

Agent Orchestration

Decomposing Complex Features

Day 1 - Session 1, Topic 1

Prerequisites

Confirm these prerequisites:

- Experience using Claude Code or a similar AI coding assistant
- Working knowledge of Git branches and merges
- Comfort using a terminal to inspect files and run commands

Introduce Yourself

In one chat message, type:

1. Your name
2. Your location or time zone
3. Your role
4. Your total experience in information technology (IT) and at your current organization

Chat Waterfall

Type your answer in the chat, but do not send it yet.

Wait for the countdown: 3... 2... 1... BLAST

You may share aloud when invited, keep a private note, or pass.

Your Agent Orchestration Experience

In one chat message, type:

1. **Your current orchestration experience** (single-agent tasks, task planning, or parallel sessions)
2. **Your main focus** (development, technical leadership, or delivery)
3. **One orchestration challenge or question** you want answered by this course

Chat Waterfall

Type your answer in the chat, but do not send it yet.

Wait for the countdown: 3... 2... 1... BLAST

You may share aloud when invited, keep a private note, or pass.

Your Agent Orchestration Experience: What We Learned

Your responses help us emphasize:

- **Your starting point:** From bounded single-agent tasks to coordinated parallel work
- **Your focus area:** Planning, implementation, technical leadership, or integration
- **Your key questions:** Task boundaries, model selection, handoffs, and verification

Compare your response with these themes and note one topic to revisit.

What We'll Cover

1. Agents, agentic development, and orchestration
2. Why complex features need decomposition
3. Agent-ready boundaries, contracts, and dependencies
4. Task cards, executable evidence, and demos
5. Lab 1.1: Agent-Ready Work Plan

What Is an Agent?

An agent works toward a goal by repeatedly deciding and acting.

1. It receives a goal, context, and constraints.
2. A model chooses the next action.
3. Tools inspect or change the environment.
4. Results become context for the next decision.
5. The loop stops when the goal is met or control returns to a human.

Agentic Development in This Course

A coding agent can inspect a repository, edit files, and run commands.

It still needs a bounded assignment, relevant context, permitted tools, and a clear definition of done.

Human responsibility: Set intent, approve consequential decisions, and judge evidence.

Agent responsibility: Perform the assigned work and report results, checks, and unresolved risks.

Skill or Agent?

REFERENCE ONLY

Extension	Primary purpose	Execution
Skill	Reusable instructions, knowledge, or workflow	Used by the current session or invoked explicitly
Agent	Specialized worker with its own context and operating boundaries	Runs an independent reasoning-and-action loop and returns a result

Use a skill when: You want Claude to follow a repeatable procedure.

Use an agent when: The work benefits from separate context, specialized tools, permissions, or parallel execution.

A skill can guide an agent, and a skill can be configured to run through a subagent. They are complementary, not competing features.

What Is Orchestration?

Orchestration is the coordination of agents, tools, and people toward a shared outcome.

- Which work belongs to each agent
- What runs sequentially or in parallel
- Which context and artifacts move between tasks
- Where verification and human approval are required

More agents do not create orchestration; explicit coordination does.

Why Decomposition Comes First

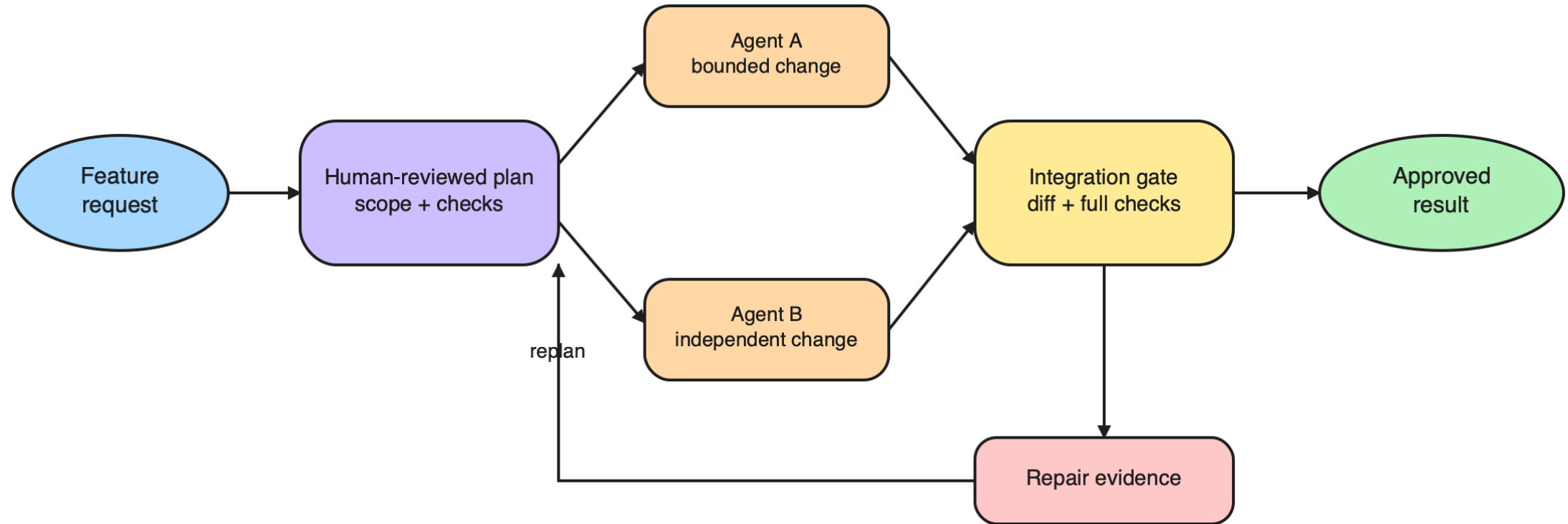
A feature is usually too broad to assign safely as one prompt.

Decomposition turns it into units with clear outputs, ownership, dependencies, and checks.

This enables parallel work where tasks are independent, limits the impact of failure, and makes progress verifiable.

Before coordinating agents, define work that can be assigned and evaluated independently.

Orchestration at a Glance



Parallel work earns its speed only when boundaries and checks are explicit.

Find Safe Boundaries

Parallelism starts with low coupling, not agent count

A Feature Is Not a Task

A feature brief states a product outcome across several concerns.

An agent-ready task produces one named, observable deliverable.

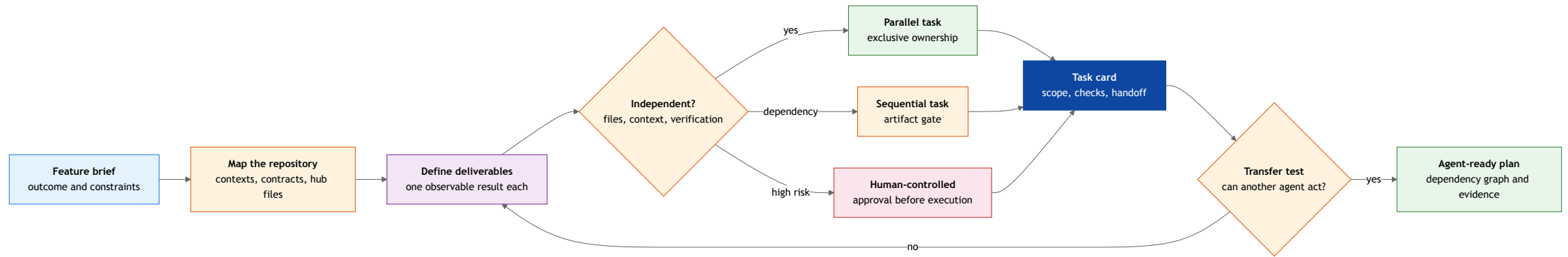
Boundary test: Can another agent act without hidden context?

Evidence test: Can another person verify completion from artifacts?

Inspect Before You Split

1. Map bounded contexts and existing contracts.
2. Find schemas, routers, manifests, and migrations.
3. Name the available verification commands.
4. Identify decisions with high blast radius.
5. Split by cohesive deliverable, not file count.

From Brief to Agent-Ready Plan



What Does the Task Card Do?

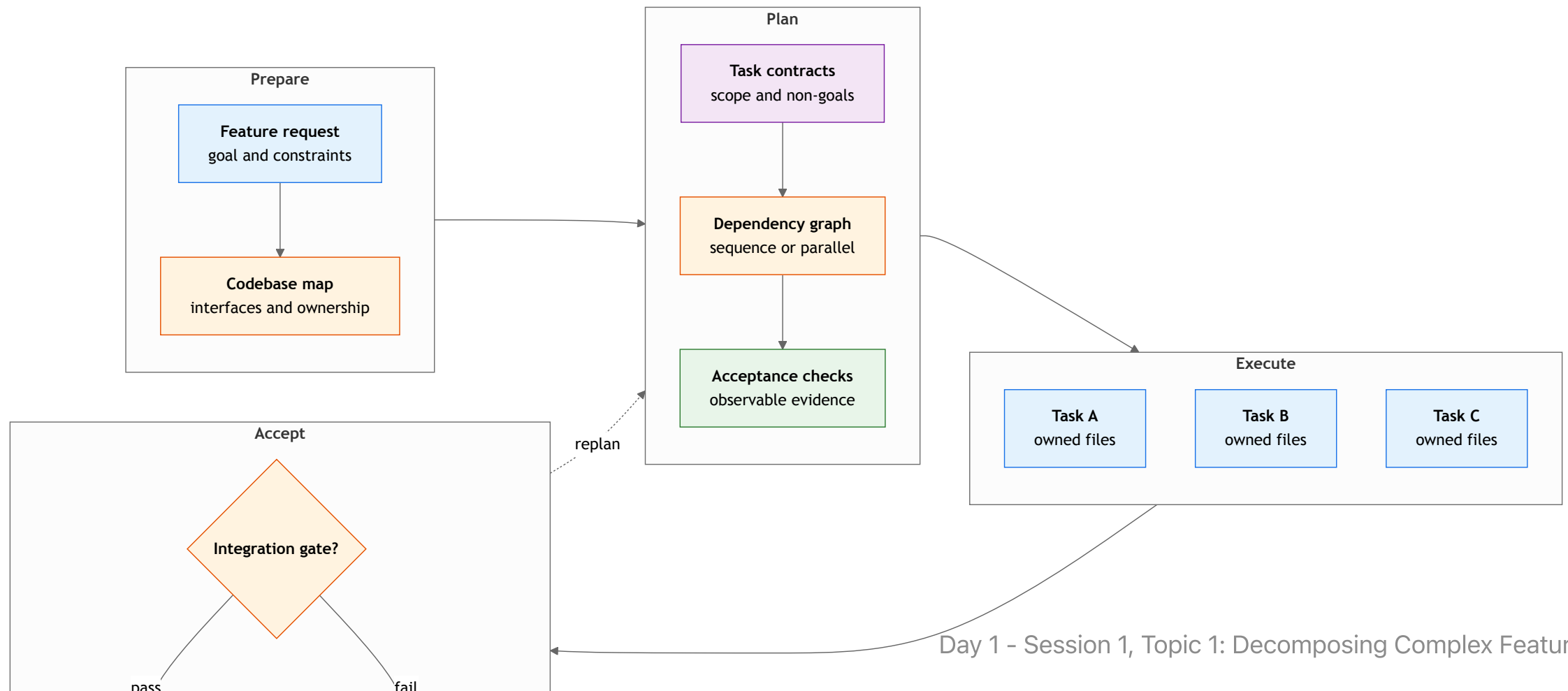
A task card turns one planned work unit into a portable assignment an agent can complete without the planning conversation.

- **Scope:** objective, deliverable, allowed files, and non-goals
- **Coordination:** dependencies and prerequisite decisions
- **Proof:** acceptance check and handoff evidence

By carrying boundaries and proof forward, the card lets the orchestrator assign work, choose safe parallelism, and verify results before integration.

Generic Feature Decomposition Flow

REFERENCE ONLY



Dependency-Aware Task Decomposition (SaaS)

SaaS: Software as a Service

REAL-WORLD SCENARIO

SaaS (Salesforce - Agentforce Vibes):

- Plan Mode breaks multi-step work into Goals, Non-Goals, dependencies, and numbered Tasks before files change.
- A human approves the plan, and a checkpoint supports rollback before execution continues.

Dependency-Aware Task Decomposition (Financial Services)

SKIP

REAL-WORLD SCENARIO

Financial services (AWS - AgentCore portfolio advisory reference):

- A portfolio query is divided among valuation, risk, market research, and advisor-synthesis agents.
- Each specialist owns one domain while an orchestrator combines results with authentication, tracing, and sandboxed code.

Enterprise Architecture Signals

Signal	Planning implication
Bounded context	Keep domain knowledge and ownership together.
Event or API contract	Approve the interface before consumers begin.
Gateway or service mesh	Keep platform operations outside feature tasks.
Security and CI gates	Automate security and independent checks before merge.
Hidden state	Expose dependencies before delegation.

Structural Hubs Change the Plan

Schemas, routers, manifests, and migrations connect otherwise separate work.

Do not: Give every channel agent permission to edit `router.py`.

Do: Lock the contract, assign channel files, and reserve integration ownership.

Shared-file conflicts often signal a missing dependency or ownership decision.

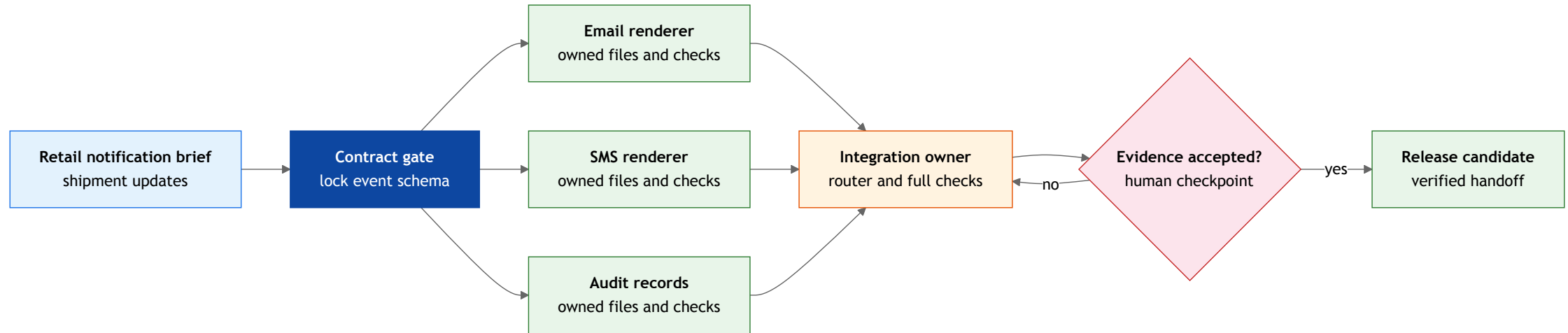
Demo: Explore Before You Decompose

From the companion repo, open `day1/demos/interactive-shipment-notification-decomposition.html`.

- Runtime flow: trace the shipment outcome across the existing architecture.
- Contract gate: identify which approval unlocks independent consumer work.
- Integration hub: find why `router.py` needs one owner and a final full check.
- Guided views: compare runtime flow, contract dependency, and integration ownership.

Self-contained Archify diagram; use Present and Live trace during the walkthrough.

Retail Notification Dependency Graph



Write Transferable Task Cards

Replace conversational assumptions with durable contracts

Task Card Contract

Field	Question it answers
Objective and deliverable	What observable result must exist?
Context and files	What may the agent inspect and change?
Dependencies	Which artifact or decision unlocks this task?
Acceptance	Which command proves the required behavior?
Non-goals	What must remain untouched?
Handoff	Which evidence transfers to the next owner?

Steps Are Not Acceptance Criteria

Weak: Add an email renderer and update its tests.

Strong: The supplied shipment fixture produces the expected subject and body when `python3 -m unittest tests.test_email` runs.

The first statement describes activity. The second defines observable evidence.

Classify the Work

Classification	Use when
Parallel	Files are exclusive, contracts are stable, and checks are independent.
Sequential	A consumer needs an upstream artifact or decision.
Human-controlled	Work is ambiguous, irreversible, sensitive, or high risk.
Not ready	The outcome, context, ownership, or verification path is missing.

Transfer Test

Give the task card to someone who missed the planning conversation.

Can they identify the deliverable, allowed files, dependencies, and command?

Can they report changed files, decisions, results, and unresolved risks?

If not, refine the card before assigning an agent.

Demo: Task Boundary Check

From the companion repo, run `python3 day1/concepts/demos-conceptual/demo-task-boundary-check.py`.

- Conflict count: naive tasks share the central router.
- Execution waves: the bounded plan exposes safe concurrency.
- Integration ownership: one task controls the hub and full check.

Runs locally with the Python standard library; no API key needed.

Demo: Task Card Transfer Test

From the companion repo, run `python3 day1/concepts/demos-conceptual/demo-task-card-transfer.py`.

- Missing fields: conversational assumptions become visible.
- Acceptance quality: vague success differs from an executable command.
- Transferability: a durable card supports independent execution.

Runs locally with the Python standard library; no API key needed.

Demo: Boundaries From the Real Import Graph

From the companion repo, run `python3 day1/demos/demo-boundary-from-imports.py`.

- Parsed graph: `ast` reads every module; `router.py` is found, not assumed.
- Hub detection: highest internal fan-out identifies the contested file.
- Waves: dependencies computed from the measured graph, then verified.

Real parsing of the course codebase; no API key, runs in a temporary copy.

Notebook: `day1/demos-notebook/demo-boundary-from-imports.ipynb`.

What the Demo Computes

The script derives file owners instead of printing a prepared judgment.

It rejects a cycle when no dependency wave can make progress.

The contract-first plan creates three waves: schema, channels, integration.

The result is evidence about the plan, not a promise that agents will coordinate.

Current Industry Evidence

Anthropic reports that multi-agent systems fit breadth-first, independent work best.

Its production guidance requires objectives, output formats, and clear boundaries.

A 2026 study reported gains from cohesion-aware partitioning over tested baselines.

Reported benchmark gains are research results, not guarantees for every repository.

Lab 1.1: Agent-Ready Work Plan

Open `day1/lab1_1_agent-ready-work-plan/` in the companion repo.

Goal: Produce a machine-checked plan for retail shipment notifications.

1. Lock the event contract and map the hub files.
2. Write bounded task cards with supplied acceptance commands.
3. Classify dependencies and record a human approval point.

This is a planning lab. Do not implement the notification system.

Key Takeaways

1. Agent-ready work has one observable deliverable and a verification path.
2. Stable contracts and exclusive ownership make parallel execution safer.
3. Dependency edges must name artifacts or decisions, not vague sequencing.
4. Human control belongs at ambiguous, sensitive, and high-risk boundaries.
5. A durable handoff carries evidence instead of full chat history.

Ready for Lab 1.1?

Build the plan before assigning the agents.